

Maximum Average Subarray of Length $\geq L$

A Complete Reasoning Trace — from wrong intuition to a binary-search feasibility check

LeetCode 644 (Maximum Average Subarray II)

This note does not just hand you the clean solution. It records the **real derivation**: how each wrong intuition arose, where it stalled, what counterexample or ground-truth check killed it, and how it was repaired. The valuable thing is not the final answer — it is the **reasoning trace**, which is exactly what actually happens in a Staff-level interview.

0. Problem

Given an integer array $a[0..n-1]$ and an integer L , find a contiguous subarray of length **at least L** whose **average is maximum**; return that average.

Constraints: $1 \leq L \leq n \leq 1e5$, $-1e4 \leq a[i] \leq 1e4$ (**negatives allowed**), answer tolerance $1e-5$. Must beat $O(n^2)$.

Let prefix sums $P[k] = a[0] + \dots + a[k-1]$, $P[0] = 0$. The average of any interval $(j, i]$ is:

$$\text{avg}(j, i] = (P[i] - P[j]) / (i - j), \text{ subject to } i - j \geq L$$

Why "length $\geq L$ " is harder than "fixed length L ": with fixed length the window is unique and a single $O(n)$ sliding window suffices; with " $\geq L$ ", for each right endpoint i the left endpoint j may range over a whole interval, giving $O(n^2)$ candidate intervals.

1. The Reasoning Trace

Attempt 1 — Window-extension pruning (the two-pointer temptation)

Intuition: fix the left endpoint, extend rightward from length L . Compare the average before and after extending by one element:

$$(a_1 + \dots + a_L) / L - (a_1 + \dots + a_{L+1}) / (L+1) = (a_1 + \dots + a_L - L \cdot a_{L+1}) / (L(L+1))$$

When a_{L+1} is large this is < 0 , prompting the thought: "the average dropped, so **stop extending**", and hence a two-pointer / monotone-extension approach.

Stall: this is only a **local comparison for one fixed left endpoint, one extension step**. Average versus window length is **neither monotone nor unimodal** — it can dip here and rebound higher two steps later. The prune misses the true optimum.

Counterexample (ground truth kills the intuition): $a = [10, -100, 10, 10, 10]$, $L = 2$.

$[10, -100]$ (avg -45) extended to $[10, -100, 10]$ (avg -26.7) is indeed lower; but extending further to $[10, -100, 10, 10, 10]$ (avg -12) rebounds. "Stop when < 0 " halts at the first step and misses the rebound.

Key takeaway. Average vs. window length is non-monotone (and not unimodal). The two-pointer prune has no valid monotone invariant to ride on; this path is dead.

Attempt 2 — The binary-search conjecture (monotonicity still unproven)

Since the average itself resists greedy reasoning, the next thought: **can we binary-search the answer?** This is only a **conjecture**. Binary search requires a monotone predicate, and "monotone in what?" is **not yet pinned down** — the detour below stems exactly from this gap.

Define $J(x)$: "does there exist a length- $\geq L$ interval whose average satisfies some relation to x ?" The key fork: should that relation be $= x$ or $\geq x$? Note this unresolved fork.

Key takeaway. Binary search is only a hypothesis here. Its validity hinges on a monotone predicate that has NOT yet been pinned down.

Attempt 3 — $g(k)$ + hashmap for equality (the key detour)

Intuition: substitute $\text{avg} = x$. If interval $(j, i]$ has average exactly x , then $P[i] - P[j] = x(i - j)$; rearranging:

$$\text{let } g(k) = P[k] - k \cdot x, \text{ then "average of } (j, i] = x" \Leftrightarrow g(i) = g(j)$$

So: scan once, store g values in a hashmap; if $g(i) = g(j)$ with $i - j \geq L$, then x is hit.

This claim is itself correct — for "does some interval average **exactly equal** x ", hash-for-equality is right. **The error is that it is useless for binary search:**

(1) binary search needs the predicate "average $\geq x$ ", not " $= x$ "; (2) " $= x$ " is a measure-zero event, almost never hit in floating point; (3) most fatally, " $= x$ " is **NOT monotone in x** (both $x_1 < x$ and $x_2 > x$ may be exactly hit by some interval), so it cannot drive a binary search.

In short: this solved a **subproblem binary search cannot use**, so it does not connect back to the main line.

Key takeaway. Hashing for $g(i)=g(j)$ correctly decides "average EXACTLY equals x ", but equality is measure-zero and, crucially, NOT monotone in x . It cannot drive a binary search.

Monotonicity debate — where the conclusion is actually forced out

Stuck after Attempt 3, all the wrestling is about "monotone in what?":

- "A pair (i, j) achieved x ; when x shrinks, must another pair (i', j') exist?" — trying to **explicitly construct** the next index pair, it cannot be done, stuck.
- "Comparing g across x at fixed indices" — this only tracks monotonicity for a fixed (i, j) ; it does not cover the optimal (i, j) **drifting with x** .
- "A judge run decides feasibility for one specific x , but the **process itself carries no transitivity**" — a decider is not a monotonicity proof; monotonicity must be proven separately.
- "We cannot set monotonicity on exact x , because both $x_1 < x$ and $x_2 > x$ may be feasible" — this precisely characterizes **why the " $= x$ " version has no monotonicity**.

The closing thought: switch the predicate to "**optimal (maximum) interval average $\geq x$** ". Let

$$f^* = \max_{i-j \geq L} \text{avg}(j, i], \text{ then } J(x) = [f^* \geq x]$$

f^* is a **constant**. $J(x) = [f^* \geq x]$ is **obviously monotone** in x : smaller x is easier to satisfy, with a unique threshold f^* , all-yes to the left and all-no to the right. **Monotonicity holds only at this moment** — it is the conclusion reached after the detour and the debate, **not a given assumed back in Attempt 2**.

Key takeaway. Monotonicity is recovered ONLY after switching the predicate to "optimal average $f^* \geq x$ ". This is the late-arriving payoff of the whole detour, not an assumption made up front.

Closing — $O(n)$ implementation of the " $\geq$ " predicate

With monotonicity established, all difficulty collapses to one thing: **given x , decide in $O(n)$ whether there exists a length- $\geq L$ interval with average $\geq x$** . Starting from the correct inequality:

$$\text{avg}(j,i) \geq x \iff P[i] - P[j] \geq x(i - j) \iff P[i] - i \cdot x \geq P[j] - j \cdot x$$

Still let $g(k) = P[k] - k \cdot x$ (note: now $\geq$, not Attempt 3's $=$). The predicate becomes:

$$\text{exists } j \leq i - L \text{ with } g(i) \geq g(j) \iff g(i) \geq \min_{j \leq i-L} g(j)$$

Key precision: what is needed is not "the smallest index j ", but the **minimum of $g(j)$ over $j \leq i-L$** . As long as $g(i)$ is no less than this rolling minimum, a valid j exists.

Implementation (single judge $O(n)$): scan i from L to n ; at each step first **fold the newly-eligible** index $j = i-L$ (its g value) into the rolling minimum m , **then** compare $g(i) \geq m$. Return true if found.

Pitfall: you must "**fold $g(i-L)$ first, then compare $g(i)$** " — the order keeps j within the valid $\leq i-L$ window; reversed, illegal j sneak in.

2. Final Algorithm (clean)

(1) Precompute prefix sums P . (2) Binary-search the candidate average x in $[\min a[i], \max a[i]]$. (3) Run the $O(n)$ judge for each x . (4) If judge is true, raise the lower bound; else lower the upper bound, until the width $< \text{eps}$.

judge(x): $m = +\infty$; for $i = L..n$: $m = \min(m, g(i-L))$; if $g(i) \geq m$: return true; return false

3. Complexity

A single judge is $O(n)$. With value range $V = \max a[i] - \min a[i]$, binary search to precision eps needs $O(\log(V/\text{eps}))$ rounds. Total:

$$O(n * \log((\max a[i] - \min a[i]) / \text{eps}))$$

Versus $O(n^2)$ brute force over all intervals, this is decisive for $n = 1e5$. (An $O(n)$ convex-hull / monotonic-queue method exists, but the binary-search predicate is the most robust and explainable route in an interview.)

4. Pitfalls

#	Pitfall	Fix
1	Treating average as monotone in window length; using extension pruning	Counterexample [10,-100,10,10,10] kills it; switch to binary search
2	Using the " $= x$ " predicate (hash-for-equality)	" $= x$ " is measure-zero and non-monotone; must use " $\geq x$ "
3	Assuming feasibility-monotonicity from the start	It holds only for "optimal average $f^* \geq x$ "; a late-arriving conclusion
4	Reading "smallest j " as smallest index	It is the minimum of $g(j)$ over $j \leq i-L$
5	Comparing $g(i)$ before folding in $g(i-L)$	Must fold first, then compare, to keep j in the valid window
6	Equality test in floating point / loose precision	Use $\geq$ with $\text{eps}=1e-6$; fix the number of binary-search rounds (e.g. 100)

5. Meta-lessons (the interview floor)

- **Let ground truth be the judge; do not trust assertions.** Non-monotonicity is nailed in one second by a concrete counterexample [10,-100,10,10,10], far better than verbal argument.

- **The detour itself has value.** The " $= x$ " hash line does not connect back to binary search, but it is exactly what forced the question "monotone in what?", ultimately steering toward $f^* \geq x$. Record the detour; on review you can see how monotonicity was forced out.

- **A counter-example of claim-first.** This trace is long partly because monotonicity was not established up front. The Staff-level delivery is: first give the 30-second high-level frame ("binary search the answer + monotone in the optimal average"), then drop into the $g(k)$ and rolling-minimum mechanism — not grope from mechanism toward frame.

- **The judging side can be wrong too.** In this trace the "hash-for-equality" claim, which is **itself correct**, was repeatedly mis-judged as wrong. The lesson is symmetric: an interviewer or a sparring partner can also err; the candidate should dare to push back with a counterexample / ground truth, and hold the convergence discipline of "answer exactly what is asked".